

Technical Appendix: Foundational Concepts and Mathematical Formulations for Quantum Power Systems Optimization

August 12, 2026

Contents

1	Introduction to the Appendix	3
2	Distributed Energy Resources (DERs) and Inverter-Based Resources (IBRs)	3
3	Newton-Raphson (NR) Algorithm & Jacobian Inversion in Power Flow	3
4	Mathematical Foundations of Quantum Simulation on Classical CPUs	4
4.1	Simulating Superposition and Gate Dynamics	4
4.2	Simulating Tunneling and Adiabatic Annealing	4
4.3	The Memory Wall: Hardware Scaling Limits	5
4.4	Strategic Value for Algorithm Development	5
5	Variational Quantum Algorithms & Circuits (VQA/VQC)	6
6	Quadratic Unconstrained Binary Optimization (QUBO) & Ising Model	7
7	Quantum Approximate Optimization Algorithm (QAOA)	7
8	Benders Decomposition & Hybrid Quantum-Classical Stochastic Unit Commitment	8
8.1	Algorithmic Flow and Global Convergence	8
8.2	Quantum Formulation Bottlenecks	8
9	The Alternating Direction Method of Multipliers (ADMM)	8
10	Hardware Topology Constraints and Minor Embedding Overhead	9
10.1	Minor Embedding and Physical Qubit Chaining	9
11	Powell-Hestenes-Rockafellar Augmented Lagrangian (PHR-ALM) & QPHR-ADMM	9

1 Introduction to the Appendix

This standalone technical reference guide details the mathematical and physical foundations of the quantum optimization paradigms, decomposition structures, continuous-to-discrete mappings, and network equations discussed in the chronological review. It serves as an algebraic bridge connecting classical smart grid operational challenges with analog and gate-based quantum computational models.

2 Distributed Energy Resources (DERs) and Inverter-Based Resources (IBRs)

Understanding the distinction between DERs and IBRs is essential when modeling modern power grid dynamics:

- **Distributed Energy Resources (DERs):** Small- to medium-scale physical assets—such as rooftop solar photovoltaics (PV), battery energy storage systems (BESS), and controllable loads—connected directly to the distribution network or located behind the customer meter. A DER is classified by its decentralized location and operational scale.
- **Inverter-Based Resources (IBRs):** Electricity generation or storage resources that couple to the electric grid through power electronic converters rather than traditional rotating synchronous machines. An IBR is classified by its physical, solid-state interface with the grid.

While many DERs (such as residential solar and batteries) are also IBRs, the terms are not interchangeable. Large utility-scale solar and wind farms operate as transmission-level IBRs but are not DERs, whereas local backup diesel generators operate as DERs using synchronous machines rather than inverters.

The aggressive addition of DERs and IBRs fundamentally shifts grid operations from a paradigm of continuous dynamic control to high-dimensional, discrete optimization. Crucial operational challenges—such as Distribution Network Reconfiguration (selecting switch states $x_{ij} \in \{0, 1\}$ to maintain a radial topology), DER Dispatch (scheduling binary charge/discharge cycles over time), and Phase Balancing—are inherently NP-hard combinatorial problems that overwhelm classical supervisory control systems.

3 Newton-Raphson (NR) Algorithm & Jacobian Inversion in Power Flow

In an alternating current (AC) power grid, active real power (P_i) and reactive power (Q_i) flowing through transmission lines depend non-linearly on the voltage magnitudes ($|V_i|$) and phase angles (θ_i) at every node:

$$P_i = \sum_{k=1}^N |V_i||V_k|(G_{ik} \cos \theta_{ik} + B_{ik} \sin \theta_{ik}) \quad (1)$$

$$Q_i = \sum_{k=1}^N |V_i||V_k|(G_{ik} \sin \theta_{ik} - B_{ik} \cos \theta_{ik}) \quad (2)$$

To solve for the unknown voltage profiles, the Newton-Raphson (NR) algorithm frames the grid as a multidimensional root-finding problem, much like linearizing a non-linear state-space model around a moving operating point. It defines active and reactive power mismatches (ΔP

and ΔQ). The linear update equation is formulated using the Jacobian Matrix (J), which maps the partial derivatives of power flows relative to voltage updates:

$$\begin{bmatrix} \Delta\theta \\ \Delta|V| \end{bmatrix} = J^{-1} \begin{bmatrix} \Delta P \\ \Delta Q \end{bmatrix} \quad (3)$$

At each iteration, the classical CPU computes the update increment by inverting the sparse Jacobian matrix, updates the voltage states, and repeats until the mismatches converge within a tight tolerance ($\Delta P, \Delta Q \leq 10^{-3}$). This matrix inversion bottleneck scales as $\mathcal{O}(N^3)$. In quantum hybrid workflows, this inversion can be dynamically accelerated using Variational Quantum Linear Solvers (VQLS).

In hybrid unit commitment loops, the classical NR power flow acts as a physical validation plant model. The quantum computer dictates the optimal generator scheduling; the classical CPU runs the NR power flow to check for grid state constraint violations. If violations occur, the CPU feeds mathematical penalties (Benders Cuts) back into the quantum optimizer as corrective feedback.

4 Mathematical Foundations of Quantum Simulation on Classical CPUs

To develop and validate novel quantum algorithms before deployment on physical Noisy Intermediate-Scale Quantum (NISQ) hardware, researchers rely heavily on classical CPU simulations. Because a classical CPU lacks physical quantum properties—such as microscopic spins or subatomic coherence—these simulations are executed entirely through dense linear algebra. In control engineering terms, the CPU simulates the quantum system by tracking the exact global probability distribution of all possible states simultaneously within a deterministic state-space model, known as a Statevector.

4.1 Simulating Superposition and Gate Dynamics

In physical hardware, a qubit exists in a superposition of states. On a classical CPU, this is tracked as a vector of complex probability amplitudes. For a single qubit, the CPU allocates a two-element state vector:

$$|\psi\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix} \quad (4)$$

where $|\alpha|^2$ and $|\beta|^2$ represent the classical probabilities of measuring the states $|0\rangle$ and $|1\rangle$, respectively, subject to the strict normalization constraint $|\alpha|^2 + |\beta|^2 = 1$.

When a quantum algorithm applies a discrete gate operation, such as a Hadamard gate (H), the CPU does not split a physical bit. Instead, it performs a deterministic discrete-time state update via standard matrix-vector multiplication:

$$H|\psi\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} \quad (5)$$

When a measurement is commanded, the CPU collapses the state by running a pseudo-random number generator weighted against the absolute squares of the vector elements to decide the final binary output.

4.2 Simulating Tunneling and Adiabatic Annealing

Quantum annealing hardware (such as D-Wave systems) utilizes physical quantum tunneling to pass directly through high-energy barriers in an optimization landscape, rather than climbing over them as seen in classical simulated annealing.

To simulate this continuous-time evolution, the CPU numerically integrates the time-dependent Schrödinger equation. For a control engineer, this is mathematically identical to simulating a continuous-time, unforced Linear Time-Varying (LTV) state-space system.

In standard quantum mechanics notation, the system’s evolution is governed by:

$$\frac{d}{dt}|\psi(t)\rangle = -i\hat{H}(t)|\psi(t)\rangle \quad (6)$$

where the total system Hamiltonian matrix $\hat{H}(t)$ smoothly transitions between the initial driver Hamiltonian ($\hat{H}_B$) and the final problem Hamiltonian ($\hat{H}_C$) over the normalized annealing schedule $s(t)$:

$$\hat{H}(t) = (1 - s(t))\hat{H}_B + s(t)\hat{H}_C \quad (7)$$

When programmed into a classical CPU solver, the abstract Dirac notation ($|\cdot\rangle$) is dropped entirely in favor of standard linear algebra and control theory notation:

$$\dot{\mathbf{x}}(t) = \mathbf{A}(t)\mathbf{x}(t) \quad (8)$$

Here, the state vector $\mathbf{x}(t) \in \mathbb{C}^{2^N}$ tracks the complex probability amplitudes of all possible generator configurations simultaneously. The time-varying dynamic system matrix is defined purely by the scaled Hamiltonian: $\mathbf{A}(t) = -i\mathbf{H}(t)$.

To compute the system’s trajectory, the CPU calculates the state transition matrix (often using matrix exponentials over small discretized time steps Δt , assuming $\mathbf{A}(t)$ is piecewise constant):

$$\mathbf{x}(t + \Delta t) = e^{\mathbf{A}(t)\Delta t}\mathbf{x}(t) \quad (9)$$

In this classical matrix formulation, what physicists call ”quantum tunneling” is completely demystified. It is simply the off-diagonal, cross-coupling terms within the state-transition matrix $e^{\mathbf{A}(t)\Delta t}$. As the CPU computes each time-step, these off-diagonal terms mathematically ”leak” numerical values (probability amplitudes) from one index of the state vector $\mathbf{x}$ to another, bypassing high-energy mathematical barriers without needing to simulate physical momentum or landscape climbing.

4.3 The Memory Wall: Hardware Scaling Limits

Because the simulation must track the probability amplitude of every possible combination of states simultaneously, scaling the system incurs an exponential memory cost. For a system of N qubits, the state vector requires storing 2^N complex numbers:

- **10 Qubits:** $\approx 1,024$ amplitudes (Kilobytes of RAM)
- **30 Qubits:** ≈ 1 billion amplitudes (Gigabytes of RAM)
- **50 Qubits:** $\approx 1.1 \times 10^{15}$ amplitudes (Petabytes of RAM)

This exponential ”Memory Wall” makes exact mathematical simulation of 50+ qubit systems physically impossible on standard classical hardware, fundamentally necessitating the transition to physical Quantum Processing Units (QPUs) for grid-scale problems.

4.4 Strategic Value for Algorithm Development

Despite the severe classical scalability limits, CPU simulation remains an indispensable tool for quantum algorithmic design. It allows researchers to rigorously prove the mathematical value of new algorithms through three key mechanisms:

1. **Verifying Logical Correctness:** Before deploying code to a noisy physical QPU, simulators provide a noise-free ground truth. If an algorithm fails to solve a 4-bus power flow or stochastic unit commitment model on a simulator, its logic is inherently flawed; if it succeeds, the mathematical control logic is perfectly sound.
2. **Proving Asymptotic Scaling ($\mathcal{O}$ Notation):** By simulating algorithms across incremental qubit counts (e.g., 4, 8, and 12 qubits), researchers can extrapolate hardware-agnostic performance curves. If a classical heuristic scales exponentially ($\mathcal{O}(2^n)$) while the simulated quantum algorithm scales polynomially ($\mathcal{O}(\text{poly}(n))$), the mathematical superiority of the quantum algorithm is definitively proven.
3. **Isolating Algorithmic Flaws vs. Hardware Noise:** Physical QPUs suffer from decoherence, gate errors, and thermal noise. Simulators cleanly decouple physical hardware limitations from theoretical algorithmic bottlenecks. This isolation enables the precise mathematical tuning of objective functions and ansatz architectures before introducing real-world physical disturbances.

5 Variational Quantum Algorithms & Circuits (VQA/VQC)

Variational Quantum Algorithms (VQAs) form the primary hybrid computing paradigm for NISQ-era (Noisy Intermediate Scale Quantum) processors. Gate-based architectures rely on *Variational Quantum Circuits (VQCs)*, which replace deep, error-prone quantum circuits with shallow Parameterized Quantum Circuits (PQCs) whose gate angles are tuned via a classical feedback loop. This architecture mimics an adaptive control system minimizing a cost function.

The execution workflow follows four sequential stages:

- **Stage A: State Preparation:** Continuous classical state data are mapped into quantum states. Common methods include *Angle Encoding* (N qubits for N features, low depth) and *Amplitude Encoding* ($\log_2 N$ qubits for N features, highly compact but requires deeper circuits).
- **Stage B: The Ansatz (PQC):** A parameterized sequence of quantum gates acts on the state, leaving rotation angles as tunable control variables θ :

$$|\psi(\theta)\rangle = \hat{U}(\theta)|0\rangle^{\otimes N} \quad (10)$$

- **Stage C: Measurement & Expectation Value:** Projective measurements are repeated over multiple runs (where a "shot" is a single physical execution of the circuit) to evaluate the expectation value of the target objective function (Hamiltonian):

$$\text{Cost}(\theta) = \langle \psi(\theta) | \hat{H} | \psi(\theta) \rangle \quad (11)$$

- **Stage D: Classical Optimization:** A classical optimizer acts as the controller, evaluating the computed cost and updating the gate parameters θ using gradient-free (e.g., COBYLA) or gradient-based (e.g., Adam) methods:

$$\theta_{\text{next}} = \text{Optimizer}(\theta, \text{Cost}(\theta), \nabla \text{Cost}(\theta)) \quad (12)$$

Practical Bottlenecks: Training deep VQCs suffers from *Barren Plateaus*—where cost gradients vanish exponentially, akin to a loss of observability or control authority in high-dimensional state spaces—and *Measurement Overhead*, where heavy statistical sampling is needed to filter quantum noise.

6 Quadratic Unconstrained Binary Optimization (QUBO) & Ising Model

A Quadratic Unconstrained Binary Optimization (QUBO) problem is an NP-hard combinatorial problem defined over binary decision variables $x \in \{0, 1\}^N$:

$$\min_{\mathbf{x}} f(\mathbf{x}) = \mathbf{x}^T \mathbf{Q} \mathbf{x} = \sum_{i=1}^N Q_{ii} x_i + \sum_{i < j} Q_{ij} x_i x_j \quad (13)$$

To execute this on physical quantum processors, the binary variables $x_i \in \{0, 1\}$ must be mapped to physical spin variables $z_i \in \{-1, +1\}$ via the transformation $z_i = 1 - 2x_i$. Substituting this derives the physical Ising Hamiltonian:

$$\hat{H}_{\text{Ising}} = \sum_{i=1}^N h_i \hat{Z}_i + \sum_{i < j} J_{ij} \hat{Z}_i \hat{Z}_j \quad (14)$$

In analog Quantum Annealing (QA), this Hamiltonian is physically manufactured on superconducting chips. The system begins in a uniform superposition under a strong transverse magnetic field ($\hat{H}_{\text{initial}}$). Over an annealing schedule $s(t)$, the transverse field decays while the problem couplings energize:

$$\hat{H}(t) = (1 - s(t)) \hat{H}_{\text{initial}} + s(t) \hat{H}_{\text{Ising}} \quad (15)$$

To a control engineer, this can be conceptualized as a physical time-varying dynamical system naturally relaxing to its lowest-energy stable equilibrium (the global minimum of its Lyapunov energy function). Through *Quantum Tunneling*, the system bypasses high mathematical energy barriers, rapidly settling into the optimal binary configuration.

7 Quantum Approximate Optimization Algorithm (QAOA)

QAOA is the digital, gate-based equivalent of analog Quantum Annealing. It simulates Trotterized adiabatic evolution using alternating parameterized layers of quantum gates. From a control perspective, this alternating sequence behaves like a discrete-time bang-bang control strategy, steering the quantum state toward the target objective.

The algorithm defines four principal components:

1. **Initial State:** Qubits are initialized in an equal superposition state: $|\psi_0\rangle = |+\rangle^{\otimes N}$.
2. **Cost Hamiltonian ($\hat{H}_C$):** The diagonal problem Hamiltonian representing the classical objective function.
3. **Mixing Hamiltonian ($\hat{H}_M$):** Drives state transitions to explore the solution space. The standard is a transverse-field X-mixer: $\hat{H}_M = \sum_{i=1}^N \hat{X}_i$.
4. **Circuit Depth (p):** The number of alternating control rounds applied. A depth- p parameterized state is prepared via:

$$|\gamma, \beta\rangle = \prod_{l=1}^p e^{-i\beta_l \hat{H}_M} e^{-i\gamma_l \hat{H}_C} |\psi_0\rangle \quad (16)$$

The quantum computer executes the forward pass, and a classical optimizer updates the variational angles γ and β iteratively until the expectation value converges.

To enforce strict physical constraints (e.g., ensuring exactly k generators are active), advanced variants replace the standard X-mixer with a *Custom Mixer* (e.g., XY-mixer), effectively restricting the quantum state's reachable subspace to strictly feasible configurations.

8 Benders Decomposition & Hybrid Quantum-Classical Stochastic Unit Commitment

Stochastic Unit Commitment (SUC) optimizes day-ahead binary generator schedules under renewable uncertainty. To manage complexity, generalized Benders Decomposition (BD) decouples SUC into a discrete Master Problem (first-stage binary decisions) and K independent continuous Subproblems (second-stage power flow feasibility).

This maps perfectly to a hybrid hardware architecture:

- **Quantum Processor (QPU):** Solves the discrete Master Problem to select optimal binary states.
- **Classical Processor (CPU):** Solves the continuous scenario subproblems in parallel to compute economic dispatch, extract dual sensitivities, and construct feedback constraints (Benders cuts).

8.1 Algorithmic Flow and Global Convergence

The iteration systematically loops between classical and quantum hardware:

1. **Lower Bound (QPU):** The Master Problem solves for scheduling statuses $\mathbf{u}^k$ and expected recourse cost Υ^k , establishing a lower bound on the objective.
2. **Upper Bound (CPU):** The CPU evaluates all grid subproblems using $\mathbf{u}^k$, calculating actual physical costs to establish an upper bound.
3. **Convergence Check:** The loop closes when the relative optimality gap $(\text{UB}^k - \text{LB}^k)/\text{UB}^k$ falls below a tolerance ϵ . Otherwise, new linear Benders cuts are appended to the QPU Master Problem for the next iteration.

8.2 Quantum Formulation Bottlenecks

Executing the Master Problem on quantum hardware introduces an iterative "Qubit Trap." Native quantum spins require the continuous variable Υ to be discretized into a binary register of size $J(k)$. Furthermore, each accumulated Benders cut requires auxiliary binary slack variables F . The total required physical qubits scale dynamically as:

$$\text{Total Qubits}(k) = NT + J(k) + (k - 1)F \quad (17)$$

On constrained hardware, this linear expansion in slack variables rapidly exceeds physical QPU capacity.

9 The Alternating Direction Method of Multipliers (ADMM)

ADMM is an optimization framework blending dual ascent with robust augmented Lagrangian methods. In optimal control, it is analogous to deploying decentralized controllers that coordinate via shared penalty signals to enforce system-wide constraints.

For a problem minimizing $f(\mathbf{x}) + g(\mathbf{y})$ subject to $\mathbf{Ax} + \mathbf{By} = \mathbf{c}$, ADMM forms the **Augmented Lagrangian**:

$$\mathcal{L}_\rho(\mathbf{x}, \mathbf{y}, \boldsymbol{\lambda}) = f(\mathbf{x}) + g(\mathbf{y}) + \boldsymbol{\lambda}^T(\mathbf{Ax} + \mathbf{By} - \mathbf{c}) + \frac{\rho}{2} \|\mathbf{Ax} + \mathbf{By} - \mathbf{c}\|_2^2 \quad (18)$$

where $\boldsymbol{\lambda}$ are Lagrange multipliers and $\rho > 0$ is a penalty parameter acting like a proportional feedback gain penalizing constraint violations.

In a quantum-classical workflow, ADMM naturally splits the variable space:

1. **Discrete Update (QPU):** The binary vector $\mathbf{x}$ (e.g., unit statuses) is solved on the quantum hardware as a QUBO.
2. **Continuous Update (CPU):** The continuous vector $\mathbf{y}$ (e.g., active power dispatch) is solved on classical solvers.
3. **Dual Update (Classical Orchestrator):** The CPU evaluates the residual tracking error and updates the dual multipliers $\boldsymbol{\lambda}$.

Crucially, ADMM avoids the massive qubit overhead associated with converting linear inequalities into exact equations via slack variables.

10 Hardware Topology Constraints and Minor Embedding Overhead

Translating theoretical QUBO models onto physical QPUs introduces a mapping bottleneck: the disparity between dense logical problem networks and sparse physical hardware wiring.

10.1 Minor Embedding and Physical Qubit Chaining

While optimization problems often feature fully connected logical graphs, physical qubits are arranged in sparse, fixed lattice topologies (e.g., D-Wave Pegasus or IBM Heavy-Hex). To execute a dense logical problem, one must perform **minor embedding**: mapping a single logical variable to a connected chain of multiple physical qubits.

To force all physical qubits within a chain to behave identically, a strong ferromagnetic coupling energy (J_{chain}) is applied. This introduces severe operational drawbacks:

1. **Quadratic Qubit Overhead:** Embedding an N -variable dense problem onto a sparse planar grid scales the required physical qubits to $\mathcal{O}(N^2)$.
2. **Dynamic Range Compression:** Devoting significant energy range to maintaining chain integrity (J_{chain}) compresses the available energy scale for actual problem optimization, amplifying hardware noise sensitivity.
3. **Chain Breaks:** If physical thermal noise overpowers J_{chain} , the physical qubits representing a single variable evaluate to conflicting states, destroying logical integrity and requiring classical error-correction heuristics.

11 Powell-Hestenes-Rockafellar Augmented Lagrangian (PHR-ALM) & QPHR-ADMM

Standard inequality constraint handling in QUBO requires logarithmic binary encoding of integer slack variables, wasting physical qubits. To achieve a slack-free formulation, the PHR-ALM method maps inequality constraints $g_i(\mathbf{x}) \leq 0$ directly into the objective function.

Conceptually similar to exact penalty methods in optimal control, PHR-ALM analyzes the first-order optimality condition of the traditional augmented Lagrangian with respect to a continuous slack variable S_i . By solving for S_i analytically:

$$S_i^2 = \max \left\{ -\frac{\lambda_i}{\sigma} - g_i(\mathbf{x}), 0 \right\} \quad (19)$$

and substituting it back into the Lagrangian, the physical slack variables are completely eliminated. This leaves a mathematically equivalent, strictly unconstrained function:

$$L(\mathbf{x}; \boldsymbol{\lambda}, \sigma) = f(\mathbf{x}) + \frac{1}{2\sigma} \sum_{i=1}^w \left([\max\{\sigma g_i(\mathbf{x}) + \lambda_i, 0\}]^2 - \lambda_i^2 \right) \quad (20)$$

This equation executes seamlessly on a Quantum Annealer without auxiliary slack qubits. The classical supervisor subsequently updates the multipliers λ and stiffness σ based on the constraint violation residuals.

When integrated as **QPHR-ADMM**, the power system master problem is decoupled generator-by-generator into independent blocks. This sequential state-updating limits the physical QPU footprint to a flat constant ($\max\{T, J\}$), completely bypassing hardware capacity limits regardless of grid scale.